Home Content Contact

C++ SYNTAX

DATA TYPES, VARIABLES, AND STRUCTURE

Start

C++ SYNTAX

- Syntax refers to the rules of writing C++ code
- Similar to grammar in human languages
- Determines whether a program is valid or invalid

ANALOGY: SPELLING/GRAMMAR in a LANGUAGE

WHY SYNTAX MATTERS IN C++

- C++ is a strict language
- Small syntax errors cause compilation errors
- The compiler does not guess or correct mistakes

Analogy: Think of a compiler as a grammar nazi

BASIC RULES OF C++ SYNTAX

- C++ is case-sensitive
- Statements usually end with a semicolon (;)
- Curly braces { } define blocks of code
- Code is read from top to bottom

WHAT IS A C++ PROGRAM STRUCTURE?

A basic C++ program has:

- Preprocessor directives
- Main function
- Statements
- Return statement

C++ PROGRAM STRUCTURE

- The majority of instructions in a program are statements
- Functions are groups of statements that execute sequentially

C++ PROGRAM STRUCTURE

- The majority of instructions in a program are called **statements**
- **Functions** are groups of statements that execute sequentially
- In programming, a single piece of data is called a **value** (sometimes called a data value)
- Values that are placed directly into the source code are called **literals**.

RANDOM ACCESS MEMORY (RAM)

- RAM is the main memory where programs are loaded and run.
- Program code and hardcoded data are loaded into RAM at startup.
- Extra RAM is used for user input, file/network data, and calculated values.
- RAM acts like numbered storage locations used temporarily while a program runs.

OBJECTS AND VARIABLES

- Direct memory access is discouraged in C++.
- Memory is accessed indirectly through objects.
- An object represents a region of memory that holds a value.
- The compiler manages where objects are stored in memory.
- A variable is a named object used to store and retrieve values.

WHAT IS A VARIABLE?

- A variable is a named storage location
- Used to store data
- The value can change during program execution

VARIABLE DEFINITION & CREATION

- When we want to give a name to a variable it is called **definition**.
- At **runtime** each object is given an actual storage location. The process of reserving storage for an object’s use is called allocation

WHAT IS A DATA TYPE?

- A data type defines the type of data
- Determines what kind of values can be stored
- Helps the computer manage memory

COMMON C++ DATA TYPES

- int – whole numbers
- float – decimal numbers
- double – more precise decimals
- char – single character
- string - text
- bool – true or false

INTRODUCTION TO IOSTREAM: COUT, CIN, AND ENDL

- input/output library (io library) is part of the C++ standard library that deals with basic input and output.
- **std::cout** allows us to send data to the console to be printed as text. cout stands for “character output”.
- To print more than one thing on the same line, the insertion operator (<<) can be used multiple times in a single statement to concatenate (link together) multiple pieces of output. For example: std::cout << "Hello" << " world!";

STD::ENDL

- To output a newline
- If we want to print separate lines of output to the console, we need to tell the console to move the cursor to the next line. We can do that by outputting a newline. A newline is an OS-specific character or sequence of characters that moves the cursor to the start of the next line.
- One way to output a newline is to output std::endl (which stands for “end line”):
- Example: std::cout << "Hi!" << std::endl;

STD::CIN

- std::cin (which stands for “character input”) reads input from keyboard.
- We typically use the extraction operator (>>) to put the input data in a variable
- Example: std::cin >> x;

KEYWORDS AND NAMING IDENTIFIERS

- C++ reserves a set of 92 words (as of C++23) for its own use. These words are called **keywords** (or reserved words)
- The name of a variable (or function, type, or other kind of item) is called an **identifier**.

alignas	const_cast	int	static_assert
alignof	continue	long	static_cast
and	co_await (since C++20)	mutable	struct
and_eq	co_return (since C++20)	namespace	switch
asm	co_yield (since C++20)	new	template
auto	decltype	noexcept	this
bitand	default	not	thread_local
bitor	delete	not_eq	throw
bool	do	nullptr	true
break	double	operator	try
case	dynamic_cast	or	typedef
catch	else	or_eq	typeid
char	enum	private	typename
char8_t (since C++20)	explicit	protected	union
char16_t	export	public	unsigned
char32_t	extern	register	using
class	false	reinterpret_cast	virtual
compl	float	requires (since C++20)	void
concept (since C++20)	for	return	volatile
const	friend	short	wchar_t
consteval (since C++20)	goto	signed	while
constexpr	if	sizeof	xor
constinit (since C++20)	inline	static	xor_eq

IDENTIFIERS NAMING RULES

- The identifier can not be a keyword. Keywords are reserved.
- The identifier can only be composed of letters (lower or upper case), numbers, and the underscore character. That means the name can not contain symbols (except the underscore) nor whitespace (spaces or tabs).
- The identifier must begin with a letter (lower or upper case) or an underscore. It can not start with a number.
- C++ is case sensitive, and thus distinguishes between lower and upper case letters. nvalue is different than nValue is different than NVALUE.

LITERALS AND OPERATORS

- A literal (also known as a literal constant) is a fixed value that has been inserted directly into the source code.

- **Operators:** In mathematics, an operation is a process involving zero or more input values (called **operands**) that produces a new value (called an **output value**). The specific operation to be performed is denoted by a symbol called an **operator**.

ACTIVITY: What would be the output

1.

```
int main(){
    string num1="20",num2="50";
    cout<<num1+num2;
}
```

2.

```
int main(){
    int num1=10,num2=30,num3=2;

    cout<<num2+num3*num1;
}
```

3.

```
int main(){
    string fname="angelo",lname="echor";

    cout<< "val " << fname << " " <<lname;
}
```

4. Assign your first name in variable fname then print it out. The output should be:
Hi! I am First Name.